```
        _, dt_old = _parse_ts(p_old["timestamp"])
        if dt_new is None or dt_old is None:
            return None

        dt = (dt_new - dt_old).total_seconds()
        if dt <= 0:
            return None

        lat1 = p_old["lat_e6"] / 1000000.0
        lon1 = p_old["lon_e6"] / 1000000.0
        lat2 = p_new["lat_e6"] / 1000000.0
        lon2 = p_new["lon_e6"] / 1000000.0
        dist_m = _haversine_m(lat1, lon1, lat2, lon2)
        speed_cms = int(round((dist_m / dt) * 100.0))
        heading_de2 = int(round(_bearing_deg(lat1, lon1, lat2, lon2) * 100.0))
        vs_cms = int(round(((p_new["altitude_m"] - p_old["altitude_m"]) / dt) * 100.0))

        return {
            "speed_cms": max(0, speed_cms),
            "heading_de2": max(0, heading_de2),
            "vs_cms": vs_cms,
        }

    def _fit_linear_regression_1d(self, xs, ys):
        n = len(xs)
        if n < 2:
            return None
        x_mean = sum(xs) / n
        y_mean = sum(ys) / n
        sxx = 0.0
        sxy = 0.0
        for x, y in zip(xs, ys):
            dx = x - x_mean
            sxx += dx * dx
            sxy += dx * (y - y_mean)
        if sxx <= 0.0:
            return None
        b = sxy / sxx
        a = y_mean - b * x_mean
        return a, b

    def _predict_from_linear_regression(self, flight_key, age_sec, fallback_lat, fallback_lon, fallback_alt_m):
        """
        Common ML baseline: sliding-window linear regression over recent trajectory points.
        Fits lat(t), lon(t), alt(t) and extrapolates for outage duration.
        """
        cur = self._conn.cursor()
        cur.execute(
            """
            SELECT timestamp, lat_e6, lon_e6, altitude_m
            FROM trajectory_samples
            WHERE flight_key = ?
            ORDER BY timestamp DESC
            LIMIT 20
            """,
            (str(flight_key).strip().upper(),),
        )
        rows = cur.fetchall()
        if len(rows) < 4:
            return None

        pts = []
        for r in reversed(rows):
            _, dt = _parse_ts(r["timestamp"])
            if dt is None:
                continue
            pts.append(
                (
                    dt,
                    r["lat_e6"] / 1000000.0,
                    r["lon_e6"] / 1000000.0,
                    float(r["altitude_m"] or 0.0),
                )
```